

Prueba Técnica — Desarrollador Fullstack

.NET + Angular

Tecnología: .NET Core (última versión) + Angular (última versión) **Plazo de entrega:** 3 días

Contexto del Problema

EventosVivos es una startup que organiza eventos culturales, conferencias y talleres. Actualmente gestionan todo mediante hojas de cálculo y formularios en papel, lo que les genera problemas constantes:

- Venden más entradas que la capacidad del venue porque no tienen control en tiempo real.
- No pueden gestionar conflictos de horarios cuando un venue tiene múltiples eventos.
- Los administradores pierden horas validando manualmente reservas y pagos.

Te han contratado para desarrollar el núcleo del sistema de reservas que resuelva estos problemas.

Requerimientos Funcionales

RF-01: Crear Evento

El sistema debe permitir crear un evento con los siguientes datos:

- **Título** (obligatorio, 5-100 caracteres)
- **Descripción** (obligatorio, 10-500 caracteres)
- **Venue** (obligatorio, referencia a un lugar preexistente)
- **Capacidad máxima** (obligatorio, entero positivo, debe ser $\leq$ capacidad del venue)

- **Fecha y hora de inicio** (obligatorio, debe ser futura)
- **Fecha y hora de fin** (obligatorio, debe ser posterior al inicio)
- **Precio de entrada** (obligatorio, decimal positivo)
- **Tipo de evento** (obligatorio: conferencia , taller , concierto)

RF-02: Listar Eventos con Filtros

El sistema debe permitir listar eventos con filtros opcionales:

- Por **tipo de evento**
- Por **fecha** (rango de inicio)
- Por **venue**
- Por **estado** (activo, cancelado, completado)
- Búsqueda por **título** (búsqueda parcial, case-insensitive)

RF-03: Reservar Entrada

Un usuario puede reservar entradas de un evento:

- Se debe especificar: **eventoId**, **cantidad**, **nombre del comprador**, **email del comprador**
- Validar que existan entradas disponibles (no exceder capacidad)
- Validar que el email tenga formato válido
- Validar que la cantidad sea 1 o más
- Crear una **reserva** con estado pendiente_pago
- **Regla:** Si el evento tiene menos de 24 horas para iniciar, solo se permite reservar máximo 5 entradas por transacción

RF-04: Confirmar Pago de Reserva

El administrador puede confirmar el pago de una reserva:

- Cambiar estado de pendiente_pago a confirmada
- Generar un **código de reserva** único (formato: EV-{6 dígitos})
- Si la reserva ya está confirmada, rechazar con error
- Si la reserva fue cancelada, rechazar con error

RF-05: Cancelar Reserva

Cualquier parte puede cancelar una reserva:

- Cambiar estado de `confirmada` a `cancelada`
- **Liberar las entradas** reservadas para que estén disponibles nuevamente
- Registrar la fecha/hora de cancelación
- Si la reserva ya está cancelada o pagada/confirmada, rechazar con error apropiado

RF-06: Reporte de Ocupación

El sistema debe generar un reporte por evento que muestre:

- Total de entradas vendidas (confirmadas)
- Total de entradas disponibles restantes
- Porcentaje de ocupación
- Total de ingresos (precio × entradas confirmadas)
- Estado del evento (activo, cancelado, completado)

Reglas de Negocio

ID	Regla	Descripción
RN-01	Capacidad del venue	Un evento no puede exceder la capacidad del venue asignado
RN-02	Superposición de venues	Dos eventos activos no pueden compartir el mismo venue con horarios superpuestos
RN-03	Restricción de horario nocturno	Eventos en weekends (sábado/domingo) no pueden iniciar después de las 22:00
RN-04	Restricción de reserva tardía	No se permiten reservas para eventos que inicien en menos de 1 hora
RN-05	Limitación de entradas por transacción	Eventos con precio > \$100 limitan a máximo 10 entradas por transacción
RN-06	Estado del evento	Un evento se marca completado automáticamente cuando la fecha actual supera su hora de fin
RN-07	Cancelación con penalización	Si se cancela una reserva confirmada con menos de 48 horas del evento, se registra como "perdida" (no se libera para venta, solo para reporte)

Datos de Referencia

Venues:

ID	Nombre	Capacidad	Ciudad
1	Auditorio Central	200	Bogotá
2	Sala Norte	50	Bogotá
3	Arena Sur	500	Medellín

Tipos de evento válidos: conferencia , taller , concierto

Requerimientos Técnicos

- La **arquitectura es de libre elección** del candidato. Se evaluará la decisión arquitectónica como indicador de capacidades de diseño.
- Se requieren **pruebas automatizadas** (unitarias como mínimo).
- La base de datos es **a elección del candidato** (en memoria, SQL Server, PostgreSQL, etc.).
- La API backend debe exponer endpoints RESTful bien diseñados.
- El frontend debe ser una aplicación Angular funcional que consuma la API.

Entregables Obligatorios

1. **Repositorio GitHub público** con el código fuente completo
2. **README.md** con:
 - Instrucciones claras para ejecutar el proyecto localmente
 - Descripción de la arquitectura elegida y justificación
 - Tecnologías utilizadas
3. **Tests automatizados** que validen los flujos de negocio
4. **URL de la aplicación desplegada** en cualquier proveedor de nube (valorado positivamente como diferenciador)

Criterios de Evaluación

Se evaluará:

- Cumplimiento de los requerimientos funcionales y reglas de negocio
- Arquitectura y diseño de la solución
- Calidad del código y principios de diseño aplicados
- Manejo de errores y excepciones
- Seguridad de la aplicación
- Cobertura y calidad de las pruebas
- Documentación

Notas Adicionales

- Puedes usar herramientas de IA (Copilot, ChatGPT, etc.) como parte de tu flujo de trabajo.
- No hay una "arquitectura correcta" única — se evalúa la **calidad de la decisión**, no la decisión en sí.
- Los casos borde y validaciones son tan importantes como los flujos principales.